



Learnable Gabor Filters in CNNs: Avoiding Filter Degeneration via Early Stopping Based on Similarity Metrics

Carlos Orozco-Solis¹, Alfonso Rojas-Domínguez¹(✉) , Héctor Puga¹ ,
Manuel Ornelas Rodríguez¹ , Martín Carpio¹ ,
and Valentín Calzada-Ledesma²

¹ Tecnológico Nacional de México, campus León, 37290 Gto., Mexico
alfonso.rojas@gmail.com

² Tecnológico Nacional de México, campus Purísima del Rincón, 36425 Gto., Mexico

Abstract. The advent of Deep Learning introduced a paradigm shift in the design and implementation of machine learning models, from the *feature engineering* paradigm towards the *feature learning* one; nowadays much less effort is dedicated to the manufacture of feature extraction methods, albeit at the expense of requiring larger volumes of training data and extended training times for deep models to learn the meaningful features. Nonetheless, it has been observed that the initial layers of many image models tend to converge to some of the earlier engineered feature extractors, mainly in the form of Gabor filters and other spatial filters, thus generating a growing interest in replacing the first layers in CNNs with learnable spatial filters, and in particular, Gabor filters. In this work we investigate the problem of parameter convergence in learnable Gabor filters, discover that the filters can exhibit degradation after a few epochs of training, and propose a method based on similarity metrics between the Gabor filters to address this issue. This research can contribute to the design of more efficient training strategies of networks employing learnable spatial filters, to leverage their intrinsic advantages over the more popular non-engineered convolutional filters.

Keywords: Learnable spatial filters · Gabor filters · Convolutional Neural Networks · Early Training-Stopping

1 Introduction

Convolutional Neural Networks (CNNs) [8] possess a powerful learning capability, based on multiple stages of feature extraction, allowing them to automatically learn complex data representations [6]. Before the popularization of CNNs, feature extraction relied on manually designed filters (spatial filters); these offer great flexibility, but these require laboriously manually tuning parameters.

It has been observed that after training CNNs there is a tendency of the convolutional filters in the first layer to converge towards spatial filters such as

Gabor filters [10]. In fact, a recent study states that “the early layer weights of diverse image models tend to converge to Gabor filters and color-contrast detectors” and “Spatially localized versions of canonical 2D Fourier basis functions, such as Gabor filters or wavelets, are perhaps the most frequently observed universal features in image models.” [12].

In other words, the same attributes that are learned by the first convolutional layers can be captured using spatial filters designed specifically for that purpose. From this observation, the perspective arises of replacing some convolutional filters with Gabor filters, due to their flexibility in analyzing and representing features in an image. This idea has led to various strategies, reviewed in Sect. 2.

This work examines a CNN architecture where the filters of the first convolutional layer are replaced with trainable or *learnable* Gabor filters, referred to as a GaborNet. In particular, the convergence of the filter parameters is analyzed and it is found that a) the filter design is decided early in the training process; b) these filters exhibit degeneration after a certain training epoch; c) a point for early stopping can be identified via metrics of filter similarity, thus avoiding their degeneration and enabling ending the training after a few epochs.

2 Related Work

To take advantage of the benefits of spatial filters, some authors have proposed applying these on training images before given to a CNN. In [7], the authors apply spatial filters to detect features in faces, improving the accuracy of a CNN by 8.5%. In [4], the authors use spatial filters for handwritten digit classification, achieving competitive results against a LeNet network [8], which obtains 99.05% accuracy, while the authors’ proposal achieves 99.16%.

The integration of spatial filters into CNNs is a recent trend that has shown to improve network performance in some cases. For example, the GCN network from [10] uses a bank of Gabor filters and achieves 99.37% accuracy on MNIST, comparable to 99.43% of the Oriented Response Network [17]. On CIFAR10/100, GCN achieves 96.12% and 79.87%, respectively, halving the required parameters compared to the Wide Residual Network [16], which achieves 96.00% and 80.95%.

To further integrate spatial filters into CNNs, it has been proposed that the first layer be composed exclusively of spatial filters, adjusted by the network in the same way as conventional convolutional layers. GaborNet [2] and PCFNet [11] employ this approach. However, the performance advantage of using spatial filters in the first layer diminishes as the size of the training dataset increases.

The same is demonstrated in tests conducted on a Predefined Filter CNN (PCF) [11], which implements spatial filters like Gabor (Ga), Sobel (So), and Schmid (Sc) in its initial layer. Using only 5% of the CIFAR10 dataset, PCF-GaSc-ResNet18 achieved an accuracy of 66.32%, surpassing ResNet18 (with 62.05%). Similarly, using only 20% of the CIFAR100 dataset, PCF-GaSc-WRN168 reached 55.15%, surpassing WRN168 (with 53.27%). However, when evaluating the complete CIFAR10/100 datasets, no significant differences were observed.

In [2], a simple CNN and AlexNet are compared with their respective Gabor variants. The results show that the Gabor-CNN outperforms the CNN by 6% accuracy on the “Dogs vs Cats” dataset. In the “AffectNet” dataset, the difference is 3%. In the “ImageNet” dataset, no significant difference is observed.

3 Methodology

3.1 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a type of deep neural network designed to process data organized in a grid, typically images. CNNs can learn hierarchical representations of data through convolutional layers [9]. The learned representations are then employed in image analysis tasks such as classification, segmentation, identity recognition, etc.

The architecture of a CNN is organized through several layers, each of which contains multiple filters or convolutional kernels. These kernels are applied to input images through convolution, an operation involving the shifting of a window over the image (1). This process facilitates feature extraction by utilizing a specific set of weights and multiplying them with the corresponding elements of the receptive field [3]. The convolution operation can be expressed as follows:

$$g(x, y) = w * f(x, y) = \sum_{i=-a}^a \sum_{j=-b}^b w(i, j) f(x - i, y - j) \quad (1)$$

where $g(x, y)$ is the filtered image, $f(x, y)$ is the original image, w is the convolution kernel, $\sum_{i=-a}^a \sum_{j=-b}^b$ denotes a double sum over all positions of the kernel, the indices i and j represent coordinates in the convolution kernel, and a and b are used to define the size of the kernel.

3.2 GaborNet Architecture

The base architecture employed in this work consists of a simple CNN composed of 3 convolutional layers, each followed by subsampling. In the first layer, 12 kernels of size 7 are used, while in the second layer, 16 kernels of size 5 are employed and finally the third layer contains 120 kernels, also of size 5. This simple architecture enables us to perform a detailed analysis that otherwise would be obfuscated by a larger and more complex CNN.

The first layer consists of 12 convolutional kernels w , each one a 7×7 matrix, implying a total of 49 parameters that the network must train for each kernel. Given the tendency of these filters to converge towards spatial patterns known as Gabor filters, replacing these w with Gabor functions is explored. Gabor functions are complex sinusoids modulated by a Gaussian envelope [2]:

$$g(x, y, w, \theta, \psi, \sigma) = \exp\left(-\frac{x'^2 + y'^2}{2\sigma^2}\right) \exp(i(wx' + \sigma)) \quad (2)$$

$$x' = x \cos(\theta) + y \sin(\theta) \quad (3)$$

$$y' = -x \sin(\theta) + y \cos(\theta) \quad (4)$$

Equation (2) can be expressed in its real and imaginary parts; in this work, we use the real part of the Gabor function:

$$g(x, y, w, \theta, \psi, \sigma) = \exp\left(-\frac{x'^2 + y'^2}{2\sigma^2}\right) \cos(wx' + \psi) \quad (5)$$

where (x, y) denotes the pixel position in the spatial domain, w represents the central angular frequency of a sinusoidal plane wave, θ indicates the counter-clockwise rotation of the Gaussian function (i.e., the orientation of the Gabor filter), and σ represents the sharpness of the Gaussian function along the x and y directions. We set $\sigma \approx \pi/w$ to define the relationship between σ and w as described in [13]. By combining multiple Gabor filters with different orientations and frequencies, what is known as a filter bank is formed.

To leverage these filters within the context of a CNN, we introduce a specialized convolutional layer designed to apply a bank of Gabor filters, through which the 49 parameters for a kernel are now reduced to just 4 parameters (w, θ, ψ, σ). Integrating this Gabor layer into the CNN yields the GaborNet architecture [2]. Notice that because we have modified the original GaborNet, and to avoid confusion, we refer to our own implementation as **GaborNet2**.

3.3 Implementation

To develop **GaborNet2** in Python, it is convenient to use the PyTorch library, which provides a dynamic interface for building and training deep learning models. With this tool, we can effectively implement a convolutional neural network (CNN) architecture in which a layer with Gabor filters is integrated.

The base architecture of the CNN in PyTorch is defined by creating a class that inherits from `nn.Module`. In the constructor of this class, the convolutional (`nn.Conv2d`), pooling (`nn.MaxPool2d`), normalization (`nn.BatchNorm2d`), and activation layers (`nn.ReLU`, `nn.Sigmoid`, etc.) are defined.

A notable feature of PyTorch is the integration of various optimizers, such as stochastic gradient descent (SGD), Adam, RMSprop, or Adagrad. These optimizers adjust the network weights to minimize a loss function through the iterative update of descending gradients automatically. For this reason, the Gabor layer was designed based on the traditional `nn.Conv2d` layer.

The convolutional layer `nn.Conv2d` is a subclass of `_ConvNd`, so the custom layer `GaborConv2d` also inherits from this. In the constructor of this custom layer, 4 main parameters of the Gabor filters are defined: spatial frequency, orientation, standard deviation, and phase. These parameters are trainable and are updated through SGD (Adam) just as the weights of a traditional layer.

Once the `GaborConv2d` layer is defined, it is easy to replace the first convolutional layer of the CNN base architecture with this custom layer. By doing so, the CNN will use Gabor filters instead of conventional filters in its initial layer.

4 Experimental Setup

The parameters and configuration used to evaluate **GaborNet2** and identify potential points for early stopping are detailed in Table 1. All experiments are repeated 50 times (i.e. crossvalidation folds) to make our conclusions as robust as possible. Also, to give greater relevance to the Gabor layer compared to the other convolutional layers, the latter remain frozen during training, preserving the initial values of all convolutional kernels.

Two types of initialization: *Independent* and *Same*, were evaluated. *Independent* means that filter $j \in 1, \dots, 12$ was randomly initialized with a different seed for each of the executions. Meanwhile, *Same* initialization means that the j -th filter was initialized using the same random seed every time, for the sake of increased repeatability and hoping to achieve a better (more clearly distinguishable) convergence of the trainable parameters. Due to space limitations only the results of the *Same* initialization are reported; however, those of the *Independent* initialization were qualitatively equivalent and lead to the same conclusions.

Datasets.- In our experiments we employ two datasets that are easily represented by spatial filters. The MNIST dataset is a large database of handwritten digits commonly used in the field of machine learning. It comprises 60,000 training images and 10,000 test images of size 28×28 pixels [1]. To make our results less dependent on the dataset, we repeated the experiments on a dataset of similar difficulty: Fashion-MNIST (FMNIST) [15], which shares the characteristics of MNIST (number of images, classes, size of images and training-test split).

Gabor Filter Parameters.- The initialization of the 4 parameters (w, θ, ψ, σ) of the Gabor layer is based on [13], as follows: $w_n = \frac{\pi}{2} \cdot 2^{\frac{-(n-1)}{2}}$, $n \in [1, 5]$; $\theta_m = \frac{\pi}{8} \cdot m - 1$, $m \in [1, 8]$; $\psi \sim \mathcal{U}(0, \pi)$; $\sigma \approx \frac{\pi}{w}$.

This configuration results in a Gabor filter bank with 5 different scales and 8 different orientations. This diversity in scales and orientations allows for optimal adaptability to a wide range of features in processed images.

Filter Similarity Metrics.- The procedure to identify an early stopping for the training consists in computing the pairwise distance between each of the filters in the Gabor layer, treated as data vectors, one iteration at a time. Several metrics can be employed. For two vectors $u, v \in \mathbb{R}^n$ with mean values $\bar{u}$ and $\bar{v}$, the following metrics were evaluated:

Entropy (η).- the entropy of a grayscale image (in this case, each Gabor filter) is a measure of randomness that can be used to characterize the image:

$$\eta = - \sum (p \cdot \log_2(p)) \quad (6)$$

where p contains the histogram counts for the input images, which were treated as grayscale images with an 8-bit depth, thus producing histograms of 256 bins.

Correlation-Based Distance.- One minus the sample correlation between points (i.e. the sequences of values in x and y):

$$d_{corr} = 1 - \frac{(u - \bar{u})(v - \bar{v})^T}{\sqrt{(u - \bar{u})(u - \bar{u})^T} \sqrt{(v - \bar{v})(v - \bar{v})^T}} \quad (7)$$

Hamming Distance.- The percentage (fraction) of elements (coordinates or dimensions) that differ between u and v :

$$d_{Hamm} = \#(u_j \neq v_k)/n, \quad j, k \in \{1, \dots, n\} \quad (8)$$

where the $\#(\cdot)$ function represents the number of occurrences for which the condition in the argument is satisfied.

Table 1. Experimental configuration

Parameter	Description	MNIST	FMNIST
Epochs	Number of training iterations	25	50
Optimizer	Optimization algorithm used	Adam	Adam
Executions	Number of repetitions of the experiment	50	50
Batch size	Training batch size	256	256
Learning rate	Learning rate	0.001	0.001

5 Results

Convergence of the parameters in the Gabor layer was tested by performing 50 executions or folds, training the **GaborNet2** and recording the value of each of the filter parameters per iteration for the purposes of observing the average behavior of the parameters over the whole set of 12 filters, during training.

Figure 1 shows the results of the filters' parameters convergence using *Same* filter initialization. For a better interpretation, the mean value was centered at zero and we plot the difference in the mean value between iterations, per filter. This processing has the effect of displaying the displacement of each parameter, irrespective of their actual value; this is shown as dashed lines in Fig. 1. Furthermore, the standard deviation is presented as shaded regions around the mean. Observe that both the displacement and the change in the dispersion are negligible. However, Loss and Accuracy plots (shown in Fig. 2) are as expected and do not allow the identification of a problem in the **GaborNet2**.

The lack of convergence of the Gabor filters led us to examine each filter as it is trained, discovering that, after a few training epochs, the filters exhibit gradual degeneration, as shown in Fig. 3. The final step in our methodology is to compute similarity metrics between the filters, following the assumption that some of these could be used to detect the point at which filter degradation began, and thus identify a good time for early stopping of the training process.

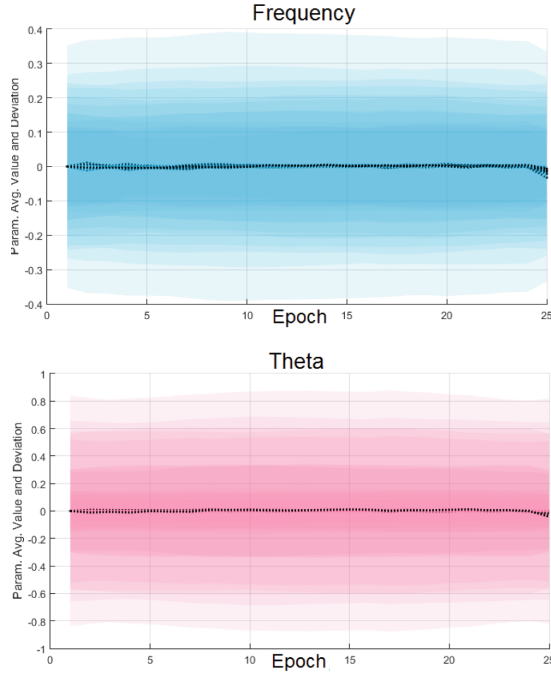


Fig. 1. Parameter convergence with *Same* filter initialization. These results are illustrative of the general behavior for all the filter parameters.

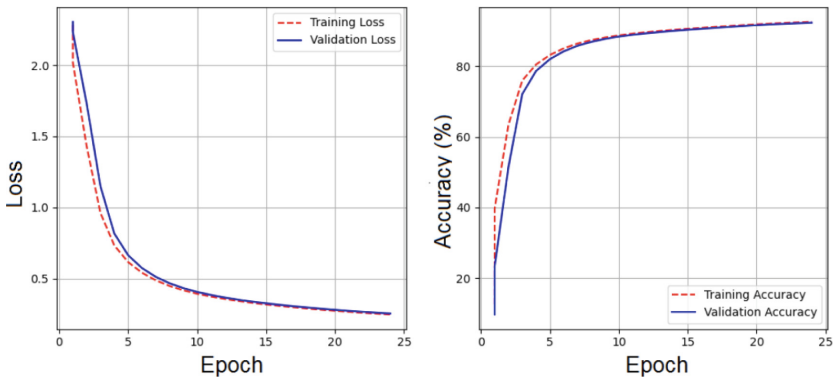


Fig. 2. Loss and Accuracy plots of training with *Same* filter initialization. These plots correspond to the MNIST dataset. Plots corresponding to FMNIST are very similar but are not shown due to space limitations.

The results of our procedure are shown in Figs. 4 5 and 6. All of the metrics tested in this experiment could allow for the identification of an “inflection” point (around epoch 16), but this is more clearly observed in the Correlation between

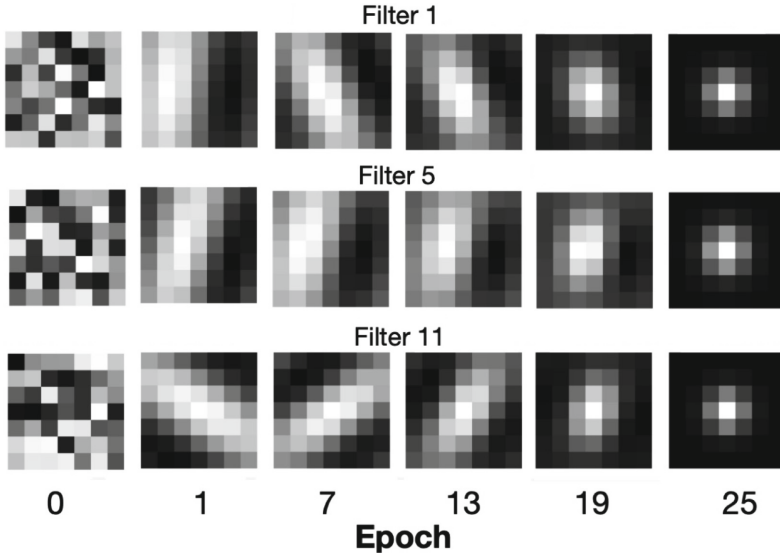


Fig. 3. Selection of 3 filters (top row: Filter 1, middle row: Filter 5, and bottom row: Filter 11) in the Gabor layer at different training epochs, illustrating gradual degeneration of the filters as the training progresses.

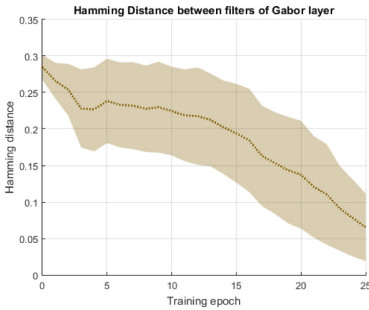


Fig. 4. Hamming distance between filters of the Gabor layer.

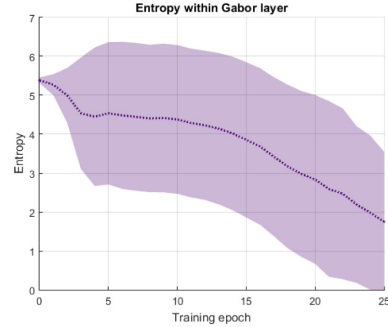


Fig. 5. Entropy of filters of the Gabor layer.

filters of the Gabor layer, shown in Fig. 6 where said point has been marked in black. This point in fact avoids significant filter degradation and, as said before, the network does not incur in any meaningful loss of performance.

In the experiments conducted on the MNIST dataset, an accuracy of 93.12% was achieved; meanwhile, evaluating the model on the FashionMNIST dataset produced an accuracy of 79.97%. A similar difference (about 10%-13%) between the results of MNIST and FMNIST has been observed before (see for instance [5]), even when the accuracy during training is well above 95%. This indicates that good generalization is harder to achieve on FMNIST.

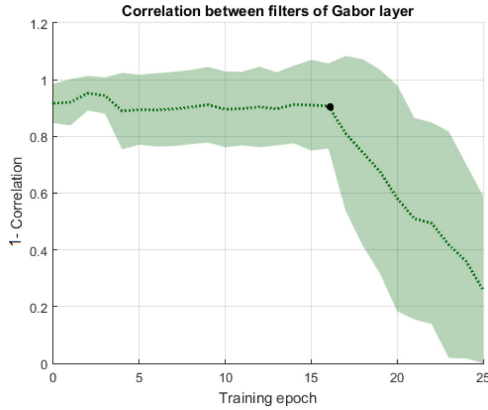


Fig. 6. Correlation-based distance between filters of the Gabor layer and inflection point (early stopping point) identified by a black dot.

6 Conclusion

Although several works have explored the use of spatial filters, and in particular that of Gabor filters [2, 4, 7, 10, 11, 16, 17], through our review of such works we found that a detailed analysis of the convergence of the filters had not been carried out. During our own analysis it was observed, not only a lack of convergence of the filter parameters, but a gradual degradation of the filters after a certain point in the training process. We have described our methodology and the proposed solution, based on the computation of a correlation-based distance metric between the Gabor filters, in order to identify the point at which training on the Gabor layer can be stopped to avoid filter degradation.

As to the reasons why the observed degradation takes place, we speculate that the Fully Connected (FC) layers have learned useful features that enable the adequate performance of the network, so that, as training continues, the network degrades the Gabor filters, as it does not need them any further. In other words, in lieu of the convolutional filters, a network can rely on many other neurons (e.g. 11,000 in the FC layers) to maintain or improve its performance. Compare this with [14], where CNN filters are compressed without significantly compromising accuracy (thus showing that not every filter is required to maintain performance).

In future work we will continue investigating the behavior of spatial filters as the initial layers of CNNs and design training strategies to leverage their intrinsic characteristics that make these desirable over the more popular kernels. Two aspects worthy of future study are the filters convergence and the decreased efficiency of GaborNet architectures when dealing with large datasets.

Acknowledgements. This work was supported by the National Council of Humanities, Science and Technology (CONAHCYT) of Mexico, via Postgraduate Scholarship 824517 (C. Orozco) and Grant CÁTEDRAS-2598 (A. Rojas).

References

1. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998)
2. Alekseev, A., Bohe, A.: GaborNet: Gabor filters with learnable parameters in deep convolutional neural network. In: 2019 International Conference on Engineering and Telecommunication (EnT), pp. 1–4. IEEE (2019)
3. Bouvrie, J.: Notes on convolutional neural networks (2006)
4. Calderon, A., Roa, S., Victorino, J.: Handwritten digit recognition using CNNs and Gabor filters. *Proc. Int. Congr. Comput. Intell.*, 1–9 (2003)
5. Kadam, S.S., Adamuthe, A.C., Patil, A.B.: CNN model for image classification on MNIST and fashion-MNIST dataset. *J. Sci. Res.* **64**(2), 374–384 (2020)
6. Khan, A., Sohail, A., Zahoor, U., Qureshi, A.S.: A survey of the recent architectures of deep convolutional neural networks. *Artif. Intell. Rev.* **53**, 5455–5516 (2020)
7. Kwolek, B.: Face detection using convolutional neural networks and Gabor filters. In: Duch, W., Kacprzyk, J., Oja, E., Zadrożny, S. (eds.) *ICANN 2005*. LNCS, vol. 3696, pp. 551–556. Springer, Heidelberg (2005). https://doi.org/10.1007/11550822_86
8. LeCun, Y., et al.: Backpropagation applied to handwritten zip code recognition. *Neural Comput.* **1**(4), 541–551 (1989)
9. Li, Z., Liu, F., Yang, W., Peng, S., Zhou, J.: A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE Trans. Neural Netw. Learn. Syst.* **33**, 6999–7019 (2021)
10. Luan, S., Chen, C., Zhang, B., Han, J., Liu, J.: Gabor convolutional networks. *IEEE Trans. Image Process.* **27**(9), 4357–4366 (2018)
11. Ma, Y., Luo, Y., Yang, Z.: PCFNet: deep neural network with predefined convolutional filters. *Neurocomputing* **382**, 32–39 (2020)
12. Marchetti, G.L., Hillar, C., Kragic, D., Sanborn, S.: Harmonics of learning: universal Fourier features emerge in invariant networks (2023). [arXiv:2312.08550](https://arxiv.org/abs/2312.08550)
13. Meshgini, S., Aghagolzadeh, A., Seyedarabi, H.: Face recognition using Gabor filter bank, kernel principle component analysis and support vector machine. *Int. J. Comput. Theory Eng.* **4**(5), 767 (2012)
14. Wang, Y., Xu, C., You, S., Tao, D., Xu, C.: CNNpack: packing convolutional neural networks in the frequency domain. In: *Advances in Neural Information Processing Systems*, vol. 29 (2016)
15. Xiao, H., Rasul, K., Vollgraf, R.: Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint: arXiv:1708.07747* (2017)
16. Zagoruyko, S., Komodakis, N.: Wide residual networks. *arXiv preprint: arXiv:1605.07146* (2016)
17. Zhou, Y., Ye, Q., Qiu, Q., Jiao, J.: Oriented response networks. In: *Proceedings IEEE Conference on Computer Vision and Pattern Recognition*, pp. 519–528 (2017)